TP2 — Robotique : Résoudre un labyrinthe avec A*

Quang-Trung Luu
quang-trung.luu@universite-paris-saclay.fr

Gif-sur-Yvette, le 13 mars 2026

1 Objectifs pédagogiques

L'objectif de ce TP est d'implémenter et d'analyser un algorithme de recherche de chemin dans un labyrinthe. À l'issue de ce TP, les étudiants seront capables de :

— modéliser un labyrinthe sous forme de graphe

— comprendre le principe de recherche bidirectionnelle

— comparer Dijkstra classique et Dijkstra bidirectionnel

— reconstruire le plus court chemin

2 Pré-requis et consignes

— Langage : Python 3

— Travail : binôme (sauf indication contraire).

— Le code doit être propre, commenté, et testable.

3 Principe du Dijkstra bidirectionnel

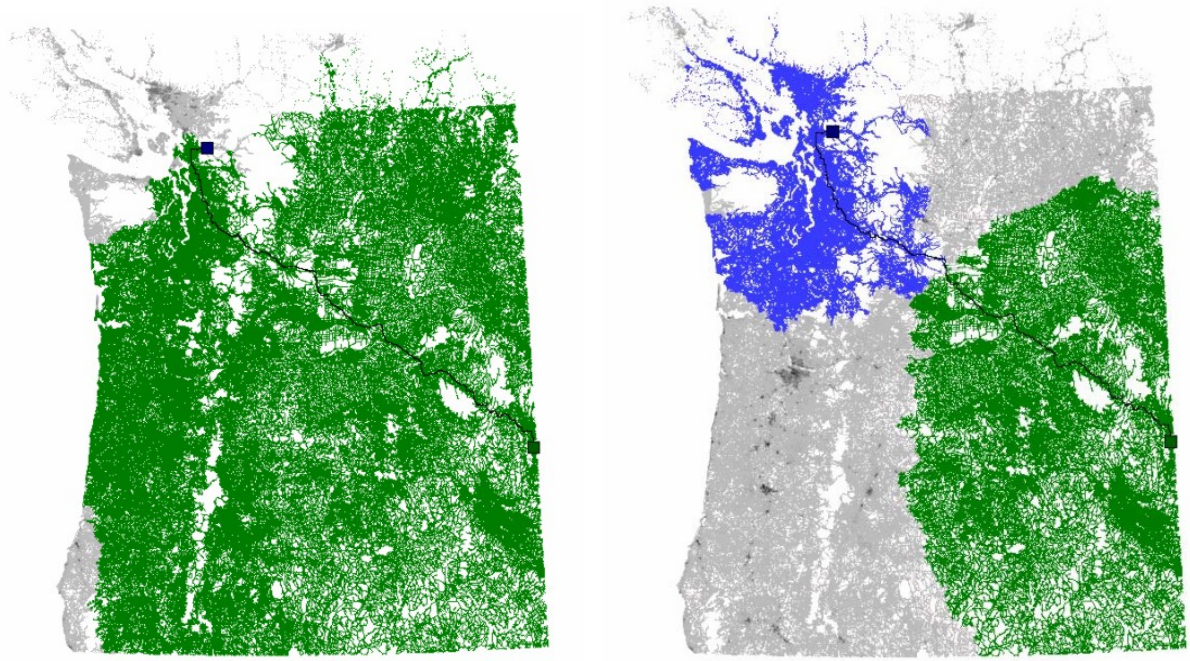
L'algorithme Dijkstra classique explore le graphe depuis la source. Le Dijkstra bidirectionnel effectue deux recherches simultanées : l'une depuis le `start` et l'autre depuis le `goal`. La recherche s'arrête lorsque les deux explorations se rencontrent, i.e., si un sommet u est atteint par les deux recherches :

$$\texttt{distance}\,(\texttt{start}, \texttt{goal}) = \texttt{distS}\,(u) + \texttt{distG}\,(u)\,,$$

où `distS` est la : distance depuis start et `distG` représente la distance depuis le `goal`.

4 Travaux et Questions

1. Implémenter l'algorithme de Dijkstra bidirectionnel. Quel est le critère d'arrêt ? Le tester sur le petit exemple donné dans le sujet.
2. Tester l'algorithme de Dijkstra bidirectionnel sur des labyrinthes de plus grande taille. Exemples :



(a) Dijkstra classique. (b) Dijkstra bidirectionnel.

FIGURE 1 – Illustration de l'algorithme Dijkstra classique et bidirectionnel (source : Goldberg et al.).

   (a) Random geometric graph : $\texttt{G} = \texttt{nx.random_geometric_graph}(2000, 0.05)$
   (b) Erdős–Rényi graph : $\texttt{G} = \texttt{nx.erdos_renyi_graph}(1000, 0.01)$
   (c) Barabási–Albert graph (scale-free) : $\texttt{G} = \texttt{nx.barabasi_albert_graph}(1000, 5)$
3. Comparer votre algorithme avec l'algorithme intégré `bidirectional_dijkstra` de NetworkX.
4. Comparer Dijkstra classique et Dijkstra bidirectionnel. Quel algorithme explore le moins de sommets ?
5. Implémenter l'algorithme A* bidirectionnel.
6. Comparer Dijkstra bidirectionnel et A* bidirectionnel. Commenter les résultats.

Références

1. A. Goldberg, C. Harrelson, H. Kaplan, and R. Werneck. Efficient Point-to-Point Shortest Path Algorithms
2. M. Rice and V. Tsotras, "Bidirectional A* search with additive approximation bounds," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 3, no. 1, pp. 80-87, 2012.
3. NetworkX's bidirectional_dijkstra : {URL}